

BÀI 11. XÂY DỰNG WEB API & XÁC THỰC VỚI JWT

Ở các chương trước, Server của chúng ta làm hai việc: xử lý logic và "vẽ" ra trang HTML. Từ tuần này, chúng ta sẽ áp dụng một triết lý mới: **Server chỉ tập trung vào một việc duy nhất và làm thật tốt: cung cấp dữ liệu và dịch vụ**. Nó sẽ không còn quan tâm đến giao diện nữa. Để làm được điều đó, chúng ta cần một ngôn ngữ giao tiếp chuẩn (RESTful API) và một phương thức "kiểm tra an ninh" không trạng thái (stateless) phù hợp, đó chính là **JSON Web Token (JWT)**.

MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài học này, bạn sẽ có khả năng:

- **Thiết kế và xây dựng** các API Controller tuân thủ theo chuẩn RESTful.
- **Phân biệt rõ ràng** sự khác biệt giữa Controller (cho MVC) và ControllerBase (cho API).
- **Giải thích và triển khai** được cơ chế xác thực bằng JWT từ đầu đến cuối.
- **Cấu hình CORS** để cho phép các ứng dụng bên ngoài (như Angular) có thể "nói chuyện" với API của bạn.
- **Sử dụng thành thạo Postman** để kiểm tra và tương tác với các API endpoint.

11.1. Giới thiệu RESTful API - Ngôn ngữ giao tiếp của Web hiện đại

11.1.1. API là gì? Người phục vụ trong nhà hàng

API (Application Programming Interface - Giao diện lập trình ứng dụng) giống như một người phục vụ (waiter) trong nhà hàng.

- Bạn (client/frontend) không cần vào bếp (backend/database) để nấu ăn.
- Bạn chỉ cần gọi người phục vụ, xem menu (tài liệu API), và đặt món (gửi request).
- Người phục vụ sẽ chuyển yêu cầu của bạn vào bếp, và mang đồ ăn (dữ liệu/response) ra cho bạn.
- API là giao diện trung gian cho phép các phần mềm khác nhau "nói chuyện" với

nhau một cách có quy tắc.

11.1.2. REST (Representational State Transfer): Bộ quy tắc ứng xử của "người phục vụ"

REST không phải là một công nghệ, mà là một **tập hợp các quy tắc kiến trúc** để thiết kế các API một cách logic và dễ đoán. Một API tuân thủ REST được gọi là **RESTful**.

- Nguyên tắc 1: Mọi thứ đều là Tài nguyên (Resources)
 - Mỗi "thứ" trong hệ thống của bạn được xác định bởi một URL duy nhất.
 - Ví dụ: /api/products (tất cả sản phẩm), /api/products/10 (sản phẩm có ID là 10), /api/orders (tất cả đơn hàng).
- Nguyên tắc 2: Sử dụng Phương thức HTTP để thể hiện hành động
 - Chúng ta dùng các "động từ" của HTTP để ra lệnh cho "người phục vụ".

Phương thức	Hành động	Ví dụ
GET	Lấy dữ liệu (Read)	GET /api/products/10 -> Lấy thông tin sản phẩm ID 10.
POST	Tạo mới dữ liệu (Create)	POST /api/products (kèm dữ liệu sản phẩm mới) -> Tạo một sản phẩm mới.
PUT	Cập nhật/Thay thế toàn bộ (Update)	PUT /api/products/10 (kèm dữ liệu mới) -> Thay thế sản phẩm ID 10 bằng dữ liệu mới.
DELETE	Xóa dữ liệu (Delete)	DELETE /api/products/10 -> Xóa sản phẩm ID 10.

- Nguyên tắc 3: Stateless (Vô trạng thái)
 - Đây là nguyên tắc quan trọng nhất. Mỗi request từ client phải chứa **tất cả thông tin** mà server cần để hiểu và xử lý nó.
 - Server **không lưu trữ** bất kỳ thông tin nào về phiên làm việc của client. Người phục vụ không "nhớ" bạn đã gọi món gì ở lần trước. Mỗi lần gọi là một lần

yêu cầu lại từ đầu. Điều này dẫn đến sự cần thiết của JWT.

11.1.3. API Controller trong ASP.NET Core

- ControllerBase và Controller:
 - Controller (chúng ta đã dùng): Dành cho MVC, kế thừa từ ControllerBase và thêm các tính năng liên quan đến Views (return View(), ViewBag...).
 - ControllerBase: Lớp cơ sở gọn nhẹ, **dành riêng cho API**, chỉ chứa các phương thức cần thiết để xử lý HTTP request và trả về dữ liệu (Ok(), NotFound(), BadRequest()).
- Attributes quan trọng:
 - [ApiController]: Một attribute "thần kỳ" tự động kích hoạt nhiều hành vi hữu ích cho API, như tự động trả về lỗi 400 Bad Request nếu model binding thất bại.
 - [Route("api/[controller]")]: Định nghĩa khuôn mẫu URL cho toàn bộ controller. [controller] sẽ được thay thế bằng tên của controller (ví dụ: ProductsApi -> api/products).
 - [HttpGet], [HttpPost], [HttpPut], [HttpDelete]: Các attribute để ánh xạ Action với các phương thức HTTP tương ứng.

11.2. JSON Web Token (JWT) - "Tấm vé vào cửa" không cần hỏi lại

11.2.1. Tại sao cần JWT? Vấn đề của ứng dụng Stateless

Như đã nói, API là stateless. Vậy làm sao server biết một request đến /api/my-orders là của người dùng nào?

- **Cookie-based Authentication (của MVC)**: Không phù hợp. Cơ chế này phụ thuộc vào việc trình duyệt tự động gửi cookie, vốn hoạt động tốt giữa trình duyệt và web server, nhưng sẽ phức tạp và không chuẩn khi client là một ứng dụng mobile hoặc một server khác.
- **JWT (JSON Web Token)**: Là giải pháp tiêu chuẩn cho thế giới stateless. Nó là một chuỗi ký tự trông như vô nghĩa, nhưng thực chất là một "tấm vé" chứa thông tin đã được mã hóa và ký tên.

11.2.2. So sánh: "Phòng gửi đồ" (Session) và "Vé sự kiện" (JWT)

- **Session/Cookie:** Giống như bạn gửi đồ ở **phòng gửi đồ**. Bạn đưa đồ, nhận một cái vé số. Lần sau quay lại, bạn đưa vé, nhân viên phải vào kho **tra cứu** sổ sách để lấy đúng đồ của bạn. Server phải tốn bộ nhớ để lưu trữ session.
- **JWT:** Giống như một vé vào cửa sự kiện đã được đóng dấu.
 1. Bạn trình CCCD (username/pass) ở quầy vé (endpoint /login).
 2. Quầy vé cấp cho bạn một tấm vé (JWT) có ghi tên bạn, loại vé (role: VIP/Admin), và quan trọng nhất là một **con dấu chống làm giả** (chữ ký bí mật).
 3. Khi bạn muốn vào một khu vực (gọi API được bảo vệ), bạn chỉ cần **giơ tấm vé** ra cho bảo vệ (middleware).
 4. Bảo vệ **không cần chạy về quầy vé hỏi lại**. Họ chỉ cần nhìn con dấu trên vé, dùng đèn soi chuyên dụng (secret key) để kiểm tra xem dấu có thật không. Nếu thật, họ cho bạn vào.
 5. **Lợi ích:** Server không cần lưu trữ bất cứ thứ gì. Việc xác thực diễn ra cực nhanh, giúp hệ thống dễ dàng mở rộng (scale).

11.2.3. Cấu trúc của một JWT: Header.Payload.Signature

Một chuỗi JWT luôn có 3 phần, ngăn cách bởi dấu chấm: xxxxx.yyyyy.zzzzz

- **Header (Phần đầu):** Chứa thông tin về loại token (JWT) và thuật toán mã hóa được sử dụng (ví dụ: HS256).
- **Payload (Phần thân):** Chứa dữ liệu (gọi là **claims**), ví dụ: sub (subject - username), exp (expiration time - thời gian hết hạn), role (vai trò người dùng),...
- **Signature (Chữ ký): Phần quan trọng nhất để bảo mật.** Nó được tạo ra bằng cách kết hợp Header, Payload và một **Secret Key** (chỉ có server biết), sau đó băm chúng bằng thuật toán đã chỉ định ở Header.
 - Khi server nhận được JWT, nó sẽ tự tính toán lại chữ ký. Nếu chữ ký mới khớp với chữ ký có sẵn, token đó là hợp lệ. Nếu client cố tình thay đổi Payload (ví dụ, tự sửa role thành "Admin"), chữ ký sẽ không còn khớp nữa và token sẽ bị từ chối.

11.3. Cấu hình JWT và CORS trong ASP.NET Core

11.3.1. CORS (Cross-Origin Resource Sharing) - "Giấy thông hành" giữa các Miền

- **Vấn đề:** Vì lý do bảo mật, các trình duyệt hiện đại áp dụng **Chính sách Cùng Nguồn (Same-Origin Policy)**. Nó cấm một trang web (ví dụ http://localhost:4200 của Angular) gửi request đến một API ở một "nguồn" (origin) khác (ví dụ https://localhost:7055 của ASP.NET Core).
- **CORS là gì?** Là một cơ chế để server API của bạn "nói" với trình duyệt rằng: "Này trình duyệt, tôi cho phép các request đến từ nguồn http://localhost:4200. Cứ để họ gọi API của tôi."
- Chúng ta sẽ cấu hình CORS trong Program.cs để "cấp giấy thông hành" cho ứng dụng Angular.

11.4. Hướng dẫn thực hành

Bước 1: Cài đặt và Cấu hình trong Program.cs

1. Cài đặt gói NuGet:

- Mở PMC hoặc Terminal và chạy:
- Install-Package Microsoft.AspNetCore.Authentication.JwtBearer
- **Giải thích:** Gói này chứa tất cả các middleware và dịch vụ cần thiết để ASP.NET Core có thể hiểu và xác thực các token JWT.

2. Cập nhật appsettings.json:

```
"Jwt": {  
  "Issuer": "https://localhost:7055",  
  "Audience": "https://localhost:7055",  
  "Key": "ThisIsASecretKeyAndItNeedsToBeLongEnoughToBeSecure12345"  
}
```

- **Issuer:** Ai là người phát hành token này (thường là domain của API).
- **Audience:** Token này dành cho ai (thường là domain của client hoặc chính API).
- **Key:** **BÍ MẬT TUYỆT ĐỐI.** Đây là "chìa khóa" để tạo và xác thực chữ ký. Nó phải đủ dài và phức tạp.

3. Cập nhật Program.cs:

```
// Thêm các using cần thiết
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);
// ...
// --- CẤU HÌNH DỊCH VỤ ---
// 1. Cấu hình CORS
builder.Services.AddCors(); // Đăng ký dịch vụ CORS
// 2. Cấu hình Authentication với JWT
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.TokenValidationParameters = new TokenValidationParameters {
            // Kiểm tra xem Issuer có hợp lệ không
            ValidateIssuer = true,
            // Kiểm tra xem Audience có hợp lệ không
            ValidateAudience = true,
            // Kiểm tra xem token đã hết hạn chưa
            ValidateLifetime = true,
            // Bắt buộc phải có chữ ký hợp lệ
            ValidateIssuerSigningKey = true,

            // Cung cấp các giá trị hợp lệ
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            // Cung cấp "chìa khóa bí mật" để xác thực chữ ký
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]
)))
        };
    });

var app = builder.Build();
// --- CẤU HÌNH MIDDLEWARE (Thứ tự rất quan trọng!) ---
// ...
app.UseStaticFiles();

// 3. Kích hoạt CORS Middleware
```

```
// Middleware này phải đứng trước UseRouting và UseAuthentication
app.UseCors(builder => builder
    .WithOrigins("http://localhost:4200") // Chỉ cho phép nguồn này
    .AllowAnyMethod()
    .AllowAnyHeader());

app.UseRouting();

// 4. Kích hoạt Authentication & Authorization Middlewares
// Luôn đặt UseAuthentication() trước UseAuthorization()
app.UseAuthentication();
app.UseAuthorization();

// ...
app.Run();
```

Bước 2: Tạo API Controller

1. Trong Controllers, tạo một Controller mới, chọn **API Controller - Empty**. Đặt tên là ProductsApiController.cs.
2. Sửa code:

```
using Microsoft.AspNetCore.Authorization; // Cần cho [Authorize]
using Microsoft.AspNetCore.Mvc;
using SportsStore.Domain;

[ApiController] // Đánh dấu đây là một API Controller
[Route("api/products")] // Định nghĩa base route
public class ProductsApiController : ControllerBase // Kế thừa từ ControllerBase
{
    private IProductRepository _repository;

    public ProductsApiController(IProductRepository repo) { _repository = repo; }

    [HttpGet] // Xử lý GET /api/products
    [Authorize] // BẮT BUỘC phải có token JWT hợp lệ mới gọi được
    public IQueryable<Product> GetProducts() {
        return _repository.Products;
    }
}
```

Bước 3: Sử dụng Postman để kiểm tra

- **Postman là gì?** Một ứng dụng desktop cho phép bạn gửi các HTTP request đến API một cách thủ công để kiểm tra xem chúng có hoạt động đúng không, mà không cần phải xây dựng giao diện frontend.
- **Kiểm tra endpoint đăng nhập:** Gửi một request POST đến `https://localhost:port/api/account/login` với body chứa username/password. Bạn sẽ nhận về một chuỗi token.
- **Kiểm tra endpoint được bảo vệ:** Gửi một request GET đến `https://localhost:port/api/products`.
 - **Lần 1 (Không có token):** Bạn sẽ nhận được lỗi **401 Unauthorized**.
 - **Lần 2 (Có token):** Vào tab "Authorization", chọn Type là "Bearer Token", dán chuỗi token bạn nhận được ở trên vào. Nhấn Send. Lần này, bạn sẽ nhận được danh sách sản phẩm.

CÂU HỎI LÝ THUYẾT

1. Hãy nêu 3 nguyên tắc chính của kiến trúc RESTful.
2. Tại sao cơ chế xác thực bằng Cookie không phải là lựa chọn lý tưởng cho các ứng dụng SPA hoặc mobile?
3. Trong 3 phần của một JWT (Header, Payload, Signature), phần nào đóng vai trò quyết định đến tính bảo mật và chống giả mạo của token?
4. Nếu ứng dụng Angular chạy ở `http://localhost:4200` bị lỗi CORS khi gọi API, bạn cần phải cấu hình ở phía Angular hay phía ASP.NET Core?

TỔNG KẾT BÀI HỌC

Bài này, chúng ta đã thực hiện một bước chuyển đổi kiến trúc cực kỳ quan trọng:

- Chúng ta đã "tách não ra khỏi cơ thể", xây dựng một **Backend API** độc lập, tuân thủ theo các chuẩn mực của **RESTful**.
- Chúng ta đã làm chủ được **JWT**, cơ chế xác thực stateless tiêu chuẩn cho các ứng dụng hiện đại, hiểu rõ cách nó hoạt động và cách cấu hình.
- Chúng ta đã giải quyết được vấn đề **CORS**, một rào cản phổ biến khi phát triển kiến trúc frontend-backend tách biệt.

Backend của bạn giờ đây đã sẵn sàng để phục vụ dữ liệu một cách an toàn cho bất kỳ client nào, mở đường cho việc xây dựng một giao diện SPA mượt mà ở tuần tiếp theo.

BÀI TẬP VỀ NHÀ

1. **Tạo endpoint lấy sản phẩm theo ID:** Trong ProductsApiController, hãy tạo một Action mới GetProduct(int id) xử lý request GET /api/products/{id}. Action này sẽ tìm và trả về một sản phẩm duy nhất dựa trên id.
2. **Xây dựng AccountApiController:** Dựa trên các buổi học trước về Identity, hãy tự tay xây dựng AccountApiController với một endpoint [HttpPost("login")] nhận LoginModel (chứa username, password), sử dụng SignInManager để xác thực, và nếu thành công, tự code logic để tạo và trả về một chuỗi JWT.
3. **Sử dụng Postman:** Thực hành lại toàn bộ quy trình: gọi API login để lấy token, sau đó dùng token đó để gọi thành công API lấy danh sách sản phẩm và API lấy sản phẩm theo ID.

TÀI LIỆU THAM KHẢO

- [1]. **Create web APIs with ASP.NET Core:** [Microsoft Docs - Create Web APIs](#)
- [2]. **JSON Web Token (JWT) authentication in ASP.NET Core:** [Microsoft Docs - JWT Authentication](#)
- [3]. **Enable Cross-Origin Requests (CORS) in ASP.NET Core:** [Microsoft Docs - CORS](#)
- [4]. **Postman Learning Center:** [Postman - Getting Started](#)